

## A1. Análisis, diseño y planeación de una aplicación móvil

El desarrollo de una aplicación móvil de alto rendimiento no comienza en la primera línea de código en Flutter, Kotlin o Swift, sino en la toma de decisiones estratégicas respecto al dominio del problema, los límites del sistema y la arquitectura base. Un fallo en la fase de planeación genera deuda técnica catastrófica, acoplamiento innecesario entre lógica de negocio y capa de presentación, e imprevistos en la escalabilidad de la infraestructura.

### A.1.1 Definición del problema y viabilidad técnica

- **Inicio del producto:** Definición del alcance clave, propuesta de valor técnico y delimitación de capacidades.
- **Evaluación de restricciones móviles:** Latencia de red, disponibilidad de sensores, perfil de la batería y almacenamiento local.
- **Criterios de elección de paradigma:** Desarrollo nativo individual (Kotlin/Swift) vs. Desarrollo nativo multiplataforma (Flutter/React Native) vs. PWA.
- **Análisis de mapeo dominio-técnico:** Transformación de necesidades de usuario en casos de uso concretos y modelos de dominio.

### A.1.2 Levantamiento de requerimientos y modelado

- **Requerimientos funcionales (RF):** Funcionalidades explícitas del sistema (autenticación biométrica, sincronización en segundo plano, procesamiento de pagos).
- **Requerimientos no funcionales (RNF):** Parámetros críticos de calidad (tiempo de respuesta < 200ms en renders, soporte *offline-first*, footprint de memoria < 120MB, cifrado AES-256 en reposo).
- **Casos de uso mapeados a flujos:** Diagramación de historias de usuario bajo el prisma de transacciones atómicas de red y estados de interfaz.

### A.1.3 Arquitectura del sistema y contratos de integración

- **Patrones arquitectónicos modernos:** Clean Architecture, MVVM (Model-View-ViewModel), BloC/Redux para gestión de estado, y Driven-Domain Design (DDD) simplificado.
- **Diseño de APIs y contratos de datos:** Formato JSON, estándares REST vs. GraphQL/gRPC, versionado de endpoints, manejo de códigos de estado HTTP y estructuras de payload optimizadas para redes móviles.
- **Estrategia de persistencia local y estado:** Selección entre bases de datos relacionales locales (SQLite/Isar/Hive), almacenamiento clave-valor (Preferences/Secure Storage) y estrategias de caché (*Cache-First*, *Network-First*, *Stale-While-Revalidate*).

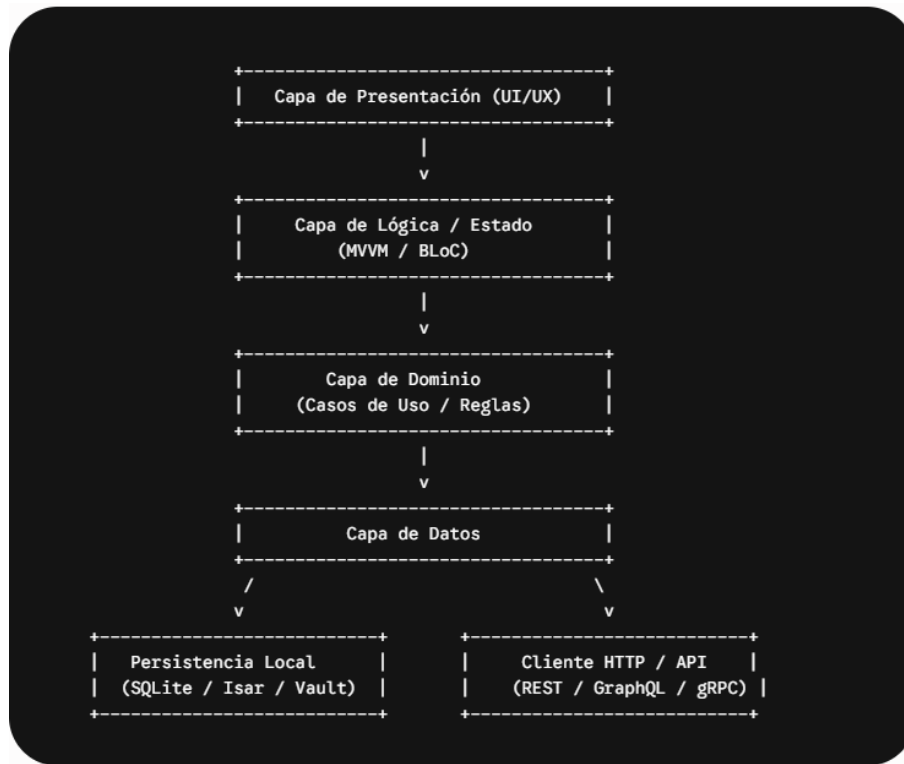
#### A.1.4 Estimación, roadmap y matriz de riesgos

- **Descomposición de tareas (WBS/EDT):** División de módulos en épicas, historias de usuario y tareas técnicas atomizadas.
- **Estimación basada en puntos de historia y riesgo:** Ponderación de complejidad algorítmica e incertidumbre de integración de terceros.
- **Evaluación de riesgos técnicos:** Dependencias con APIs externas, tiempos de aprobación en tiendas (App Store / Google Play Store), fragmentación de hardware e incompatibilidad de versiones del SO.

#### A.1.5 Fundamentos teóricos y términos clave

- **Arquitectura offline-first:** Enfoque de diseño donde la aplicación es completamente funcional sin conexión a red. Asume que la red es un estado opcional e intermitente. Operaciones de escritura se persisten en una cola local (*Outbox Pattern*) y se sincronizan contra el servidor mediante background workers cuando la conectividad se restablece.
- **Clean architecture:** Separación de responsabilidades en capas concéntricas organizadas por el Principio de Inversión de Dependencias. La capa interna (Dominio) desconoce por completo la capa externa (UI, Frameworks, Bases de Datos). Esto garantiza que los cambios en las herramientas de desarrollo o capas visuales no rompan las reglas de negocio.
- **Patrón DTO (Data Transfer Object):** Objeto que transporta datos entre procesos o capas sin lógica de negocio. En entornos móviles, es indispensable separar el DTO retornado por la red de las entidades del dominio interno para mitigar rupturas por cambios en esquemas de backend.

- **Estrategia de caché SWR (Stale-While-Revalidate):** Algoritmo donde la app sirve de forma inmediata los datos en caché (*stale*) para una experiencia visual instantánea, mientras inicia de manera asíncrona una petición de red para obtener datos actualizados (*revalidate*) y mutar la vista sin bloquear al usuario.



### A.1.6 Ejemplos prácticos y casos de estudio

#### Ejemplo A: Mapeo de requerimiento a estructura de arquitectura

*Escenario:* Módulo de captura de inventario en almacenes con conectividad deficiente.

- **Requerimiento no funcional:** La app debe registrar lecturas de código de barras a una velocidad mínima de 1 lectura por segundo independientemente del estado del Wi-Fi.
- **Diseño arquitectónico:**
  1. La UI envía el payload al *Repository*.
  2. El *Repository* guarda directamente en la BD local de alta velocidad (*Isar/SQLite*) y emite un estado de éxito inmediato a la UI.
  3. Un proceso en segundo plano (*WorkManager / Background Task*) consume la cola local pendientes de sync de la base de datos.

4. El worker realiza reintentos exponenciales (*Exponential Backoff*) al sincronizar contra la API REST central.

### **Ejemplo B: Definición de Contrato JSON para Optimización Móvil**

Evita sobrecargar el canal de datos enviando datos redundantes.

*Anti-patrón (Estructura inflada/no desacoplada):*

```
{
  "user_id": 1092,
  "user_full_legal_name": "Ozzy Smith",
  "internal_system_audit_logs": ["created_at_2026", "modified_by_admin"],
  "account_status_description_text": "Active Account"
}
```

Práctica Adecuada (Payload atómico móvil):

```
{
  "id": "usr_1092",
  "name": "Ozzy Smith",
  "status": "ACTIVE"
}
```

## **A.1.7 Actividades prácticas**

### **Actividad 1: Modelado de arquitectura e inserción de límites**

Diseñar el flujo de arquitectura y estrategia de sincronización para una app móvil de gestión de incidencias de seguridad en campo.

1. Tomen el requerimiento: *"El usuario debe poder capturar una fotografía de una vulnerabilidad física, agregar una descripción y enviarla inmediatamente al servidor o guardarla si no hay red."*
2. Dibujen la interacción entre las 4 capas principales: UI -> Controller/ViewModel -> UseCase -> Repository -> (Local DB / Network API).
3. Especifiquen por escrito la política de manejo de errores si el payload de la foto falla a mitad de la subida.

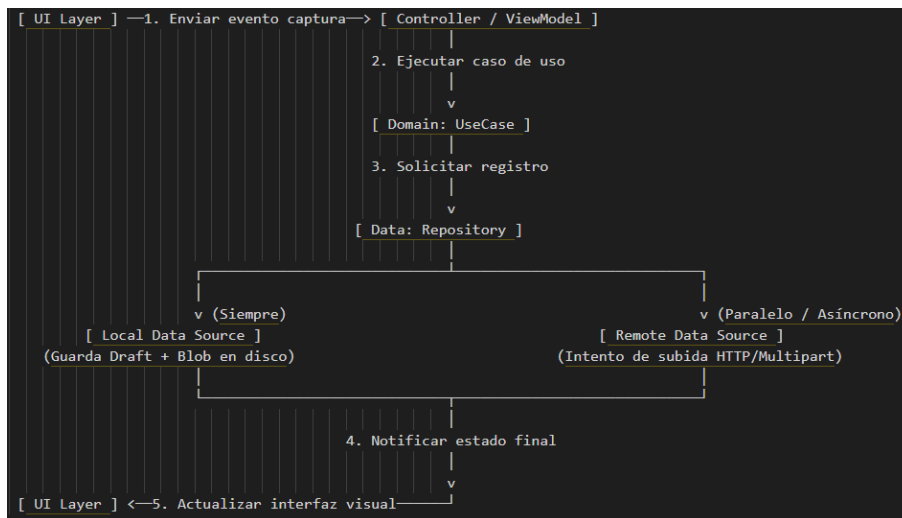
### **Solución:**

Flujo paso a paso entre capas

1. **Capa de presentación (UI):** Captura el evento de clic del usuario con la ruta de la imagen local (path) y la descripción textual. Transfiere esta información al controlador.
2. **Capa de lógica de estado (Controller/ViewModel):** Mantiene el estado de la vista (Idle, Loading, Success, Error). Invoca al caso

de uso correcto pasando un objeto de dominio con los datos procesados.

3. **Capa de dominio (UseCase):** Valida las reglas de negocio en memoria (e.g., tamaño de descripción mayor a 10 caracteres, existencia física del archivo local). Si la validación es correcta, delega el procesamiento a la interfaz abstracta del repositorio.
4. **Capa de datos (Repository - Implementación Concrete):**
  - **Paso A (Persistencia inmediata - *Offline first*):** Convierte el objeto de dominio en un DTO local y registra la incidencia en la base de datos local (SQLite/Isar) con el estado SyncStatus.PENDING. Copia la imagen a un directorio persistente seguro del dispositivo.
  - **Paso B (Intento de red):** Evalúa la conectividad mediante un NetworkChecker. Si hay red realiza una petición POST de tipo multipart/form-data.
  - **Paso C (Resolución de estado):** Si la API responde 201-Created, actualiza el registro local a SyncStatus.SYNCED. Si la API falla o no hay red, conserva el estado PENDING para su posterior sincronización en segundo plano.



Política de manejo de errores e interrupción de carga

1. **Manejo de transacciones incompletas (Fallo a mitad de upload):**
  - **Estrategia de fragmentación (Chunked Uploads) / TUS Protocol:** Si la imagen excede los 5 MB, la app fragmenta el archivo en bloques de 1 MB. En caso de corte de red en el bloque 3, el servidor conserva el token de sesión de carga y la app reanuda únicamente desde el bloque 4.

- **Identificador único causal (Idempotencia):** Cada payload generado localmente incluye un UUID v4 (client\_assigned\_id). Esto previene la duplicación de registros en caso de que la imagen llegue al backend pero la respuesta 201 se pierda por un *timeout* en la conexión descendente.
2. **Mecanismo de recostura en segundo plano (*Outbox Pattern*):**
- Ante una falla HTTP (4xx/5xx) o de socket, el registro permanece como SyncStatus.PENDING con un contador de reintentos (retry\_count = 1).
  - Se registra un trabajo persistente mediante el gestor del sistema operativo (WorkManager en Android / BGAppRefreshTask en iOS) configurado con la restricción NetworkType.CONNECTED.
  - **Estrategia de reintentos:** Se aplica un algoritmo de *Exponential Backoff con Jitter*

$$(Tiempo = Base \cdot 2^{reintento} + random)$$

Si tras 5 reintentos continúa fallando por un error 4xx (p. ej., token de sesión expirado o validación de payload fallida), el estado cambia a SyncStatus.NEEDS\_USER\_ACTION y se solicita intervención manual al usuario.

